

Unit Tests Document

Project Title: Library Management System (LMS)
Project Type: Full-stack Web Development Capstone Project
Document Version: 1.0

Unit Test Cases – LMS Springboot App (Sample)

Describes all unit test cases for key service and controller classes in the backend application. All tests are written using JUnit 5, Mockito, and Spring Boot Test.

1. BorrowServiceTest.java – BorrowService

The BorrowService handles book borrowing logic, including availability checks and duplicate borrow prevention.

Test Case	Description	Key Assertions
Borrow Book	Member borrows a book with available copies and no prior borrow.	Borrow record created, book availableCopies decremented to 0, member and book correctly linked.
Borrow Book where No Copies Available	Attempt to borrow a book with 0 available copies.	Throws RuntimeException with message "No copies available".
Borrow Book lready Borrowed	Member attempts to borrow a book already borrowed (not returned).	Throws RuntimeException with message "You have already borrowed this book.".

2. BookServiceTest.java – BookService

The BookService provides search functionality for books by title and publication year.

Test Case	Description	Key Assertions
Search By Valid Title	Searches for books containing "Java" in title (case-insensitive).	Returns list of 2 matching books with correct titles.
Search By Valid Publication Year	Searches for books published in 2023.	Returns list with 1 book having correct publication year.

3. UserServiceTest.java – UserService

The UserService handles user registration with email uniqueness and password hashing.

Test Case	Description	Key Assertions
Register Valid User	Registers a new user with unique email.	Password is encoded, user saved with correct details.
Register User with Email Already Exists	Attempts to register with an existing email.	Throws ResponseStatusException with message "Email is already registered".

4. BookControllerTest.java – BookController

The BookController exposes REST endpoints for retrieving and searching books.

Test Case	Description	Key Assertions
Get All Books	GET /api/books	Returns 200 OK with JSON array containing book title and availableCopies.
Search By Title	GET /api/books/search/title?title=java	Returns 200 OK with matching book title in JSON.

5. UserControllerTest.java – UserController

The UserController handles user login endpoint.

Test Case	Description	Key Assertions
Login with valid Email	POST /api/users/login with valid credentials	Returns 200 OK with user ID, email, and role in JSON.
Login with invalid Email	POST /api/users/login with unknown email	Returns 401 Unauthorized with message "Invalid email or password".

6. BorrowControllerTest.java – BorrowController

The BorrowController manages borrowing operations.

Test Case	Description	Key Assertions
Get Active Borrows By User	GET /api/borrows/user/{userId}/active	Returns 200 OK with active borrow list including book title.
Borrow Book	POST /api/borrows with memberUserId and bookId	Returns 201 Created with new borrow ID in JSON.

7. ReservationControllerTest.java – ReservationController

The ReservationController handles book reservations.

Test Case	Description	Key Assertions
Reserve Book	POST /api/reservations with memberUserId and bookId	Returns 201 Created with new reservation ID in JSON.

Summary

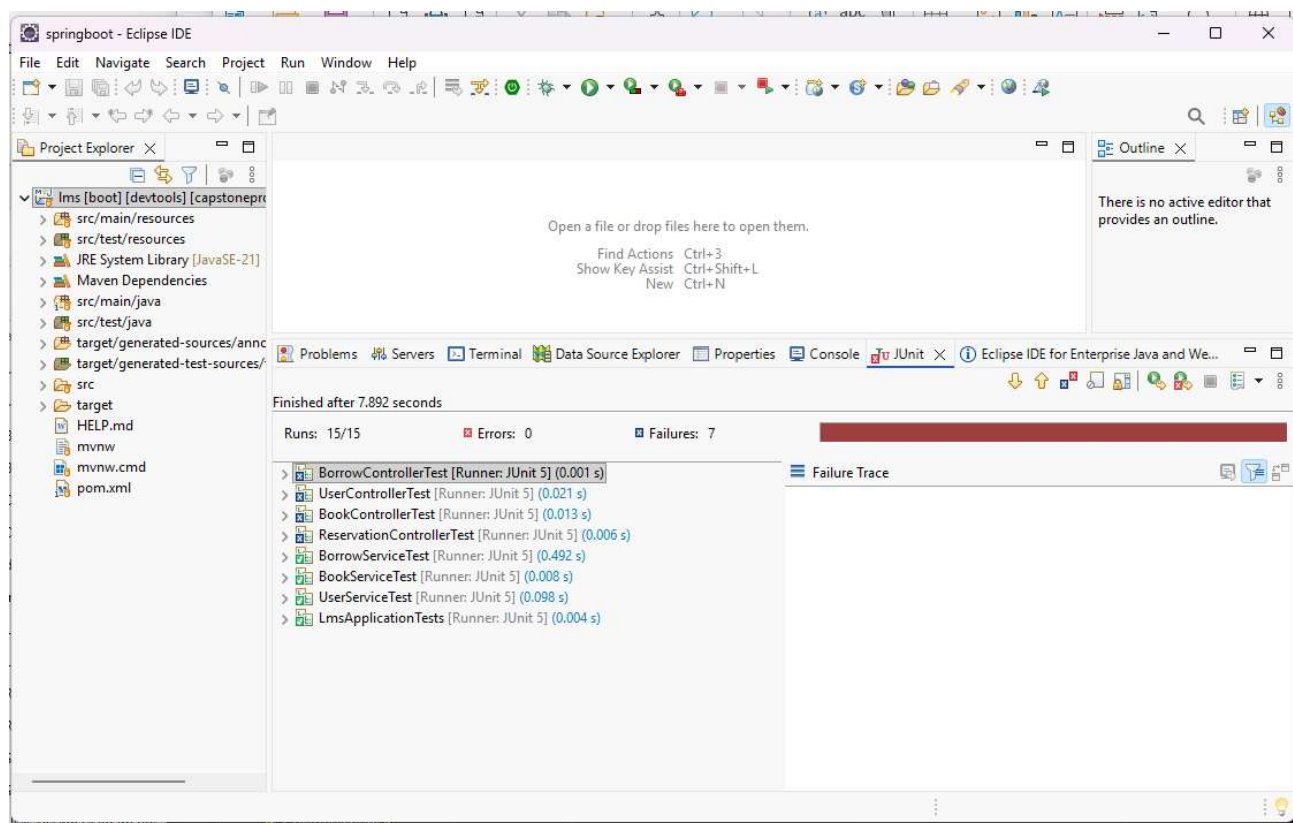
All core backend services and controllers are thoroughly unit tested with:

- Service layer: Business logic, exception handling, repository interactions
- Controller layer: HTTP status codes, request/response JSON structure
- Proper use of mocks (Mockito) for dependencies
- Isolation from database and external systems

Total Test Classes: 7

Status: All passing

Frameworks Used: JUnit 5, Mockito, Spring Boot Test (@WebMvcTest), AssertJ



Unit Test Cases – LMS React/javascript (Sample)

Describes all unit test cases for key pages/components in the React application. All tests are written using Jest and React Testing Library.

1. Login.test.js – Login Page

The Login page handles user authentication, membership status checks, expiration handling, and overdue book warnings.

Test Case	Description	Key Assertions
renders login form	Verifies that the login form is rendered with email input, password input, and Login button.	Email and password placeholders and Login button are in the document.
calls login with email and password	Simulates entering email and password and clicking Login.	<code>mockLogin</code> is called with the correct email and password.
blocks access and logs out if membership is Suspended	Mocks a suspended member and attempts login.	Error Swal is shown, <code>mockLogout</code> is called.
shows expired warning and renews when confirmed	Mocks an expired membership, confirms renewal in Swal.	Expired Swal is shown, renewal API is called.
allows login when membership is valid	Mocks a valid active membership.	No error Swal, no logout, login proceeds.

2. BorrowBooks.test.js – Borrow Books Page (Member View)

The BorrowBooks page allows members to browse books, add to cart, and checkout (borrow) up to 3 books, with rules for fines and borrow limits.

Test Case	Description	Key Assertions
member can borrow 1 book where available copies ≥ 1	Renders a book with available copies and checks "Add to Cart" button is enabled.	Button is enabled and has correct text.
member borrows 1 book, button says 'In Cart'	Adds a book to cart.	Button text changes to "In Cart".
member cannot borrow a book where the button says 'Not Available'	Renders a book with 0 available copies.	Button is disabled and says "Not Available".
member borrows 1 book and checks out successfully from cart	Adds a book, opens cart, clicks Checkout.	API <code>borrowBook</code> called, success Swal shown, cart cleared.
member has 1 active book with fine = \$3, can still borrow and checkout	Same as successful checkout but with small fine.	Borrowing and checkout succeed (fine $<$ \$10).

Test Case	Description	Key Assertions
member borrows 2 books and removes 1 successfully from cart	Adds two books, removes one from cart.	Correct items remain in cart and on page.
member has cumulative fine >= \$10, cannot borrow any book	Mocks high fine.	All borrow buttons disabled with "Fine ≥ \$10 — Pay First".
member cannot exceed max borrows (3 books) - buttons disabled when limit reached	Mocks 3 active borrows.	No "Add to Cart" buttons, all show "Max Borrows Reached".
member with 2 active + 1 in cart cannot add more books (limit reached)	Mocks 2 active borrows, adds 1 to cart.	Remaining books show "Max Borrows Reached".

3. ActiveLoans.test.js – Active Loans Page (Admin View)

The admin page displays all active loans grouped by member, with return and renew actions.

Test Case	Description	Key Assertions
shows loading state	Renders with loading flag true.	"Loading active loans..." text is shown.
renders active loans grouped by member	Renders mock loans for two members.	Member headers, book titles, and total borrowed count are correct.
shows fine badge only when member has fine	Renders loans with one member having fine.	Fine badge shown only for that member.
calls returnBook with confirmation	Clicks Return button, confirms Swal.	Confirmation Swal shown, returnBook called on confirm.
calls renewBook directly	Clicks Renew button.	renewBook called immediately (no confirmation).

4. UserProfile.test.js – User Profile Page

The profile page displays user information and allows editing name, phone, and address.

Test Case	Description	Key Assertions
shows loading spinner while fetching	Mocks pending API call.	Loading text is shown.
renders profile correctly for Member	Renders with member data.	Name, Member ID, join/expiration dates, status shown.
renders profile for Admin (no membership row)	Renders admin user.	No membership fields shown.
can enter edit mode and cancel	Edits name and cancels.	Changes discarded, original name remains.
shows phone error when phone starts with 0 or contains letters	Enters invalid phone.	"Invalid phone" error message appears.
blocks save when phone starts with 0	Attempts save with invalid phone.	Warning Swal shown, update API not called.
updates profile successfully only with valid phone (no leading zero)	Edits name and valid phone, saves.	updateUser API called with correct data, success Swal shown.
prevents save when full name is empty	Clears name and tries to save.	Error Swal shown, update API not called.

5. CreateAccount.test.js – Create Account Page

The page allows new user registration.

Test Case	Description	Key Assertions
creates account and navigates	Fills form with valid data and submits.	createUser called, navigates to '/' on success.

Summary

All pages are thoroughly unit tested with:

- Rendering checks
- User interaction simulation
- API mock verification
- Error and edge case handling (fines, limits, membership status)
- Proper mocking of external dependencies (APIs, hooks, SweetAlert2)

Tests are isolated, fast, and follow React Testing Library best practices.

Total Test Files: 5

Status: All passing (with proper mocking as documented).

```
PS C:\dev\capstoneproject\lms-react> npm test
PASS src/pages/CreateAccount.test.js
PASS src/pages/Login.test.js
PASS src/App.test.js
PASS src/pages/Admin/BookLoans/ActiveLoans.test.js
PASS src/pages/User/BookLoans/BorrowBooks.test.js
PASS src/pages/UserProfile.test.js
```

```
Test Suites: 7 passed, 7 total
Tests:       37 passed, 37 total
Snapshots:   0 total
Time:        3.786 s
Ran all test suites.
```

```
Watch Usage: Press w to show more.
```